Synchronization & Shared-State Safety

Zephyr RTOS Intermediate Course

Quick Recap — Lecture 1

- Execution contexts: threads vs ISRs
- Thread lifecycle: Ready / Running / Waiting / Suspended
- Scheduler mechanics and priorities
- Cooperative vs preemptive execution
- Priority inversion and latency

Today's Agenda

- Race conditions & shared-state hazards
- Mutexes and ownership semantics
- Semaphores and signaling patterns
- Atomic operations
- Interrupt locking & spinlocks
- Deadlocks and concurrency design

Concurrency Problems

Shared Mutable State


```
Thread A → shared memory ← Thread B
                  ↑
                 ISR
```

- Multiple execution contexts access the same data
- Execution order depends on scheduling and timing
- ISRs may interrupt execution at arbitrary moments
- **Result:**
  - Non-deterministic behavior
  - Timing-sensitive failures
  - Difficult debugging

Race Conditions

```
void foo() {
    static int counter = 0;
    counter++;                          // Looks like one operation
    // Actual steps
    // load    →   modify   →   store
    // (load 5)    (add 1)     (store 6)
}
```

```
Thread A      Thread B      counter
load 5                       5
              load 5         5
+1 → 6                       5
              +1 → 6         5
store 6                      6
              store 6        6   ← B overwrites A; one increment lost
```

DEMO 1: Shared Counter Corruption

Solution

Synchronization Toolbox

Primitive	Purpose
Mutex	Exclusive ownership of shared data
Semaphore	Event signaling / resource counting
Atomic	Lock-free single-word updates
`irq_lock()`	Prevent ISR preemption on current CPU
Spinlock	Very short busy-wait synchronization

- **Design Principle:**
  - The safest shared state is a state that is never shared

Mutexes

Mutex Fundamentals

A mutex provides **mutual exclusion** for shared resources, ensuring only one thread at a time can access a protected resource

- **Key APIs**:
  - `k_mutex_init(&mutex)`
  - `k_mutex_lock(&mutex, timeout)`
  - `k_mutex_unlock(&mutex)`
- **Properties:**
  - Ownership-based
  - Lock count (Reentrant - the owner can lock it multiple times)
  - Multiple threads may queue up waiting
  - Not usable in ISR context

Typical Usage Pattern

```
k_mutex_lock(&mutex, K_FOREVER);
shared_state++;
k_mutex_unlock(&mutex);
```

Locking API

```
k_mutex_lock(&mutex, K_FOREVER);   // blocks indefinitely (Waiting state)
k_mutex_lock(&mutex, K_NO_WAIT);   // returns -EBUSY if unavailable
k_mutex_lock(&mutex, K_MSEC(100)); // returns -EAGAIN on timeout
```

Unlocking

```
k_mutex_unlock(&mutex);  // releases ownership, wakes highest-priority waiter
```

Priority Inheritance

- **Situation: Priority Inversion** High-priority work is indirectly blocked
- **Solution:** Priority Inheritance
  - Kernel temporarily boosts priority of mutex owner
  - Owner executes at higher priority to finish critical section
  - Priority restored after unlock or timeout
- **Configuration**

```
CONFIG_PRIORITY_CEILING = -128   // -128 = unlimited
```

DEMO 2: Mutex Protection

Semaphores

Semaphores Fundamentals

A semaphore is a kernel object that implements a **counting synchronization primitive** used for signaling, coordination, and resource counting

- **Key APIs**:

  `k_sem_init(&sem, initial, limit)`

  `k_sem_take(&sem, timeout)`

  `k_sem_give(&sem)`

- **Core Properties:**
  - **Count** – number of available “permits” (0 = unavailable)
  - **Limit** – maximum value the count can reach
  - **Wait behavior** – threads may block when count is 0
  - **No ownership** – any thread or ISR may give or take (with context rules)
  - **ISR-safe to give** – `k_sem_give()` may be called from ISR; `k_sem_take()` must not block in ISR

Producer-Consumer Pattern

- **Producer**

```
k_sem_give(&data_sem);
```

- **Consumer**

```
k_sem_take(&data_sem, K_FOREVER);
```

- **Behavior**
  - Producer signals availability (increments count)
  - Consumer waits efficiently when count is 0
  - When signaled, consumer resumes execution
  - Multiple consumers may wait simultaneously

DEMO 3: Producer / Consumer Synchronization

Lightweight Synchronization

Atomic operations

Atomics perform **single, indivisible** operations on shared variables without using locks

- **Core Operations:**

```
atomic_t counter = ATOMIC_INIT(75);
atomic_inc(&counter)
atomic_dec(&counter)
atomic_add(&counter, n)
atomic_get(&counter)
atomic_set(&counter, n)
```

- **Key Properties:**
  - Never causes a context switch
  - Can be used both in threads and ISRs
  - Full memory barrier
  - Suitable for flags, counters, bitmaps

Interrupt Locking


```c
unsigned int key = irq_lock();
...
// very short critical section (no waiting, no mutexes)
...
irq_unlock(key);
```

- Disables all interrupts on the current CPU
- No ISR can run during the locked section
- Increases interrupt latency (keep it very short)

Spinlocks


```
k_spinlock_key_t key = k_spin_lock(&lock);
...
// extremely short section
...
k_spin_unlock(&lock, key);
```

- CPU spins in a tight loop until it becomes free
- The thread does not sleep. It actively waits
- Mostly a kernel / low-level driver tool

Putting It All Together

Choosing the Right Tool

Need	Primitive
Counter / flag / bit update	**Atomic**
Protect shared data	**Mutex**
Signal event or work	**Semaphore**
Very short SMP critical section	**Spinlock**
Prevent ISR interference briefly	**irq_lock()**

When Progress Stops

Circular wait

```
Thread A: owns lock1, waits for lock2
Thread B: owns lock2, waits for lock1

→ neither thread can proceed
```

- **Prevention:**
  - Always acquire multiple locks in the same global order.

Home task

- **Goal**
  - Observe and fix a real race condition
- **Task**
  - Create two equal-priority threads
  - Both increment a shared counter
  - Observe inconsistent results
- **Fix** using mutex
- Push tag: `l2-task1`

Thank You

Resources

1. Mutexes
2. Semaphores
3. Atomic APIs
4. Spinlock APIs